FRED Buffer Guide: Contents

Introduction

This manual is intended to explain the basics of writing FRED buffers. We assume that you have already learned the basics of using FRED as outlined in the FRED Tutorial Guide. It will also be helpful if you can refer to other FRED documentation. Other FRED pages give more detailed explanations of most of the things we discuss here.

This manual is divided into two parts. The first part presents a number of FRED commands that weren't described in the tutorial guide. These commands are seldom used in normal interactive text-editing, but they are important when writing buffer programs. The second part of this guide presents a number of FRED buffer programs which demonstrate how to write and use FRED buffers.

A FRED buffer (or buffer program) is really a very simple thing. Anyone who has used FRED at all knows that a buffer is simply a place for storing text. The text that you store in the buffer can be the source for a Fortran program, the contents of a company's annual report, or anything else you choose. When you write a buffer program, the text that is stored in the buffer consists of a number of FRED commands; after all, FRED commands are really just strings of characters like any other text.

Once you have the commands stored in the buffer, you can have FRED execute the commands, just as if you typed them in directly from the terminal. Of course, the whole advantage is that you *don't* have to type them in directly from the terminal. Not only does this avoid a lot of repetitive typing, but it avoids the constant risk of making a typing mistake that will foul things up. When you store FRED commands in a buffer, you can edit those commands just like any other text. In this way, you need never execute the commands until you are sure they have been entered correctly.

It is important to recognize what buffer programs can and can't do. FRED is an excellent language for manipulating text. It also has limited arithmetic capabilities and control structures (e.g. jumps and "until" loops). However, FRED is no replacement for programming languages like Fortran, APL, or Pascal; it was not designed for "number-crunching" or making complex logical decisions. Still, FRED can be enormously useful in a wide variety of applications, as will be shown in the rest of this manual.

The Condition Register

The condition register is an internal feature of FRED that can be set to either TRUE or FALSE. A number of commands set the value of this register. For example, the S command will set the condition register to TRUE

if it successfully makes a substitution; if it fails to make a substitution, it will set the condition register to FALSE. The R and W commands set the condition register to TRUE if the read or write was successful; if there was some kind of access or I/O error, the condition register is set to FALSE.

A number of commands can test the condition register to see if it is TRUE or FALSE. For example, the JT command jumps to a new line if the condition register is TRUE. In this way, you can control the order in which your program executes statements.

The T Command

One of the most common processes in FRED is examining a line of text to see if it contains a certain string of characters. When you are using FRED interactively, you can just look at the line; when you are writing a FRED buffer, you can use the Test command. The Test command has the form

```
t/<pattern>/
```

where <pattern> is any valid FRED pattern. If the current line contains a string that matches <pattern>, the condition register is set to TRUE; otherwise, it is set to FALSE.

You can also specify a range of addresses for the T command as in

```
1,$t/ $/
```

The above command tests to see if any of the lines in the current buffer end in a blank character. The current line pointer "." will be set to the first line in the range where the pattern is found.

A variation on the Test command has the form

```
T~/<pattern>/
```

This will set the condition register to TRUE if the <pattern> is NOT found in the current line; otherwise, the condition register is set to FALSE. This form of the Test command can also take a range of addresses as in

```
$-1,$t~/xxx/
```

The "T~" command will set the current line pointer "." to the first line in the range which does NOT contain a string matching the given pattern.

Normally, the Test command searches for its pattern beginning at the first address in its range and ending at the second. You can have this search go backwards by reversing the order of the addresses. For example,

```
$,1t/:/
```

begins at the last line in the buffer and searches backwards for a ":". Thus it will set the current line pointer "." to the last line in the buffer which contains a ":".

As you will see in the sections to come, Test commands are used very frequently in buffer programs.

The Jump Commands

The FRED Jump commands are similar to GOTO's in other programming languages. The various kinds of Jumps are described in the following sections.

The JM (Jump Message) Command:

The Jump Message command simply treats whatever follows it as a message that should be printed to the terminal. For example,

```
jm/Hello there/
```

prints "Hello there" out on the terminal. In place of the "/" characters, you may use any non-alphanumeric characters to delimit the message.

The JM command puts a new-line character on the end of its message. The JP (Jump Prompt) command does exactly the same thing as JM except that it omits the new-line character. For example,

```
jp;Do you want a manual (yes or no)?;
```

will print

```
Do you want a manual (yes or no)?
```

and will then wait for input without going to a new line. This is handy when you are prompting the user for input.

Line Jumps:

One of the simplest kind of Jump commands is the "line jump". These commands tell FRED to ignore any other commands which follow on the same line, depending on the value of the condition register. JT (Jump if True) tells FRED to ignore the rest of the line if the condition register is TRUE; JF (Jump if False) tells FRED to ignore the rest of the line if the condition register is FALSE. For example, consider the following line:

```
r /file1 jt r /file2
```

The first R command attempts to read a permanent file named "/file1". If the read succeeds, the condition register will be set to TRUE; because the condition register is TRUE, the JT will have FRED ignore the second Read. However, if the first read fails, the condition register will be set to FALSE; FRED will not jump at the JT and will therefore attempt the second Read. Thus the statement above will read a permanent file named "file1" if one exists; otherwise, it will look for "file2".

We should point out that the above read command with the JT would only be useful in a buffer. If a Read command fails in interactive mode, FRED issues an error and stops immediately without executing the rest of the line. If a Read fails while executing a buffer, you will not receive the usual error message; FRED will just set the condition register to FALSE and continue executing commands.

Below we give an example of the JF command.

```
*t/%/ jf jm?This buffer has a "%" character.?
```

The above Test command tests every line in the buffer for a "%" character. If it doesn't find any, it sets the condition register to FALSE and the JF skips the JM command; if there is a "%" in the buffer, the condition register is set to TRUE, the JF does not jump, and the JM prints out the given message.

Labelled Jumps:

A FRED label has the form

```
@(<name>)
```

where <name> abides by the same restrictions as buffer names. Like buffer names, the parentheses are not needed in label names if the name is only one character long and the O-I(option is in effect.

The command

```
J(<name>)
```

jumps to the label "@(<name>)". The label must be in the same buffer as the J command, but it can come before or after the command in the buffer. The search for a label begins at the line immediately following the J instruction, goes to the bottom of the buffer, and wraps around to the top if necessary. Thus a label search works the same as an ordinary pattern search. The command

```
J(<name>)T
```

will jump to "@(<name>)" if the condition register is TRUE and

```
J(<name>)F
```

will jump if the condition register is FALSE. For example, consider the following sequence of commands.

```
r /file1 j(go)t
r /file2 j(go)t
jm:Can't read either /file1 or /file2.:
q
@(go)
<more commands here>
```

The first line tries to read "/file1"; if that succeeds, the condition register is set to TRUE and FRED will jump down to "@(go)". Otherwise, FRED goes to the second line and tries to read "/file2"; if this read succeeds, the condition register will be set to TRUE and FRED will also jump to "@(go)". If neither "/file1" nor "/file2" can be read, the JM will print out its message and the Q command will tell FRED to quit. This sort of sequence is often found in FRED buffers.

The JE (Jump Exit) Command:

The Jump Exit command is a combination of the Jump Message and the Q command, to some extent. Like the JM command, it has the ability to print out a message; however, once this message has been printed, the JE command terminates execution of all buffers. Usually this means that FRED will go back into interactive mode and will take its commands directly from the terminal; however, if the O+Q option is in effect, the JE command will leave FRED entirely and return to system level. For example,

```
r /myfile jt je/Cannot read file./
```

will attempt to read "/myfile". If the read is successful, the condition register is set to TRUE and the JT will jump to the next line. However, if the read fails, the JE will be executed. The message will be printed out and FRED will go back to getting its commands from the terminal rather than the buffer program. If you wanted this kind of error to result in leaving FRED altogether, you could say

```
o+q
r /myfile jt je/Cannot read file./
```

There are several other kinds of Jump commands available in FRED. See the FRED Reference Manual for details.

The U (Until) Commands

The Until commands are repetition commands. They can be used to execute loops and conditional loops.

The simplest form of the Until command is

```
U<number> <list of commands>
```

This executes the <list of commands> a total of <number> times. For example,

```
u2 s-1/"/'/
```

will change the last two double quotes on a line into single quotes. This would change

```
"Hello," he said, "how are you?"
```

into

```
"Hello," he said, 'how are you?'
```

Any number of valid FRED commands can appear in the <list of commands>. Note though that the list only extends up to the first unescaped new-line character. If you want new-line characters in <list of commands>, you will have to escape them by preceding them with "\C", as in

```
u2 jm;She loves you;\c
   jm;Yeah, yeah, yeah;
```

The U2 loop includes both JM commands and thus will print

```
She loves you
Yeah, yeah, yeah
She loves you
Yeah, yeah, yeah
```

Note that you could have done the same thing with the command

```
u2 jm/She loves you/ jm/Yeah, yeah, yeah/
```

Another Until command is UT (Until True). This has the form

```
UT <list of commands>
```

This will set the condition register to FALSE and will then repeat the <list of commands> until the condition register is found to be TRUE at the end of one of the repetitions. Similarly, the UF (Until False) command will set the condition register to TRUE and will repeat a list of commands until the condition register is FALSE at the end of one of the repetitions. For example,

```
uf s1/.@// t/.@/
```

operates on the current line and removes all instances where the "@" character follows a non-@ character. This sort of instruction is handy in cases where you have been mistakenly using the "@" as a character deletion character. The S command in the UF loop deletes the first instance of any character followed by an "@". The T command then tests to see if there are any more "@" characters in the line. If so, the condition register is set to TRUE and the loop is executed again; if there are no more (non-@,@) pairs, the condition register is set to FALSE, and the UF will NOT repeat the loop again. If the above UF were applied to the line

```
Hellab@@o thr@ere.
```

it would loop through three times, giving

```
Hella@o thr@ere.
Hello thr@ere.
Hello there.
```

Note that we could have also written

```
uf s1/.@//
```

and left it at that; this would keep on looping through until it attempted to make the substitution but failed to find any patterns matching ".@". The S command would then set the condition register to FALSE and the loop would stop. Note that this would entail one more repetition of the UF loop than the version with the T command in it.

The final version of the Until command is UE (Until Error). This will execute its list of commands until an error occurs. Note that "unable to access file" and "no text changed" are not considered errors in the context of UE; both of these may be detected by checking the value of the condition register, and thus are not regarded as "genuine" errors. We will not say more about the UE command in this guide.

The U commands all operate in the same manner as the Global command: the <list of commands> are copied into a hidden buffer and the buffer is then executed. Because of this copying process, you will have to be careful when using stream directives in U commands. For further details, see the section on Stream Directives.

Number Registers

In order to perform simple arithmetic operations, FRED allows you to define number registers. Every number register has a name that abides by the restrictions on buffer names and label names.

Number registers are manipulated using the N command. This command has the form

```
N(<name>)<option>
```

where <name> is the name of the number register and <option> tells what you want to do with the register. Some of the possible options are listed below.

:nnn
: assigns the number "nnn" to the register. For example, "n(xx):5" assigns a value of 5 to "xx".

+nnn
: adds "nnn" to the register. For example, "n(i)+1" adds one to "i".

-nnn
: subtracts "nnn" from the register.

*nnn
: multiplies the register by "nnn".

/nnn
: divides the register by "nnn". This is standard integer division.

%nnn
: yields the value of the register modulo "nnn".

<nnn
: compares the value of the register to "nnn". If this value is less than "nnn", the condition register is set to TRUE; otherwise, it is set to FALSE.

>nnn
: compares the value of the register to "nnn". If this value is greater than "nnn", the condition register is set to TRUE; otherwise, it is set to FALSE.

=nnn
: compares the value of the register to "nnn". If the two are equal, the condition register is set to TRUE; otherwise, it is set to FALSE.

P
: prints the contents of the register. For example, "n(xx)p" prints the value of "xx".

A
: assigns the line number of the current line to the register. For example, "$n(end)a" sets "end" to the number of lines in the buffer. Note that if you specify a single address for the A option of the N command, the current line pointer "." is set to that address. If you specify two addresses, the current line pointer is set to the SECOND address and the number register is set to the line number of the FIRST address. This feature is sometimes useful when you are trying to get the line numbers at the beginning and ending of a portion of text.

The number registers can be used to perform simple integer arithmetic. They can also be used to store information that will be used later in the program.

In addition to the normal number registers, there is a special number register that is represented by the "#" symbol. This is known as the "count" register. The value of the count register may be printed with the command

```
#
```

The count register is set by a number of commands. For example, the S command sets the count register to the number of substitutions that it makes. Thus the command

```
s/./&/ #
```

will print out the number of characters in the current line. Why? "s/./&/" changes every single character in the line into itself (remember that a "&" on the right hand side of an S command stands for the string that matched the substitution pattern). Since there is a substitution for every character on the line, "#" is set to the number of characters on the line. If you wished to assign this number to a register, you could say

```
n(xx):#
```

You can also use the symbol "$" as the number of the last line of a buffer. Thus

```
n(end):$
$n(end)a
```

both assign the line number of the last line in the buffer to the number register "end". The difference is that the first form does not change the value of the current line pointer "." while the second form sets "." to the last line in the buffer.

Below we show a useful command that uses number registers.

```
g/^/ s/./&/ n(xx):# n(xx)>72 jf p
```

This command prints every line in the current buffer that contains more than seventy-two characters. The "g/^/" ensures that the list of commands will be executed for every line in the buffer. The S command assigns the number of characters in the line to the count register and the "n(xx):#" saves this number in "xx". The next command tests the value of "xx". If "xx" isn't greater than 72, the condition register is set FALSE and the rest of the line is skipped; otherwise, the condition register is set TRUE, the JF does not jump, and the line is printed.

Stream Directives

When you use FRED in interactive mode, FRED usually gets all its input from the terminal. This input can be thought of as a *stream* of characters which FRED interprets as commands, typed-in text, and so on. A stream directive is a simple way of telling FRED to obtain its input temporarily from some different source.

The simplest example of a stream directive is "\N". The construction "\N(<name>)" tells FRED to obtain its input temporarily from number register <name>. For example,

```
n(vv):5
jm/The value of "vv" is \n(vv)./
```

would print out

```
The value of "vv" is 5.
```

When FRED sees the "\N", he knows that he is to pick up input from a number register rather than the terminal. It goes to the register, finds that it contains a 5, and therefore uses the 5 where the "\n(vv)" was found. You could also say something like

```
a
\n(vv)
\f
```

Again, when FRED sees the "\N", he takes input from the number register rather than from the terminal. The lines above would append the number 5 to the current buffer.

Another simple stream directive is "\W". When FRED sees this symbol, he replaces it with the file name that is associated with the current buffer. This is very convenient, especially in system commands. For example,

```
!pascal \W
```

will be sent to TSS by FRED because of the "!" on the front. Before FRED sends off the command, it will replace the "\W" with the file name that is associated with the current buffer. Thus the command above can be used to submit a Pascal source file for compilation as soon as it is edited. This can save a substantial amount of typing, especially if you are working with files that are deep in a catalog structure.

The most commonly used stream directive is probably "\B". This tells FRED to obtain its input from an existing buffer rather than the terminal. For example, suppose buffer "a" contains the text

```
s/molecular/atomic/ p
```

Then typing

```
\b(a)
```

has exactly the same effect as typing in "s/molecular/atomic/ p". When FRED sees the "\B", he simply goes and gets his input from the buffer rather than the terminal. Thus

```
a
\b(a)
\f
```

would append the S and P commands to the current buffer. Typing "\b(a)" as a command would replace "molecular" with "atomic" in the current line.

There are several obvious advantages to using the "\B" instruction. It's usually much shorter to type than the entire contents of a buffer, and the savings in time can add up if you have to execute the same instruction repeatedly. Furthermore, you can always edit the contents of a buffer to make sure that there are no typing mistakes. If you are typing in complex patterns or complex sequences of instructions, this can be extremely important -- if you make a mistake while typing a command into a buffer, you can always fix it; if you make a mistake while typing a command directly to FRED, you may foul things up before you have a chance to correct yourself.

The simplest way to execute a buffer program is with "\B". All you have to do is type your commands into a buffer (say "buff"), and then execute the buffer by saying "\B(buff)" as a FRED command. The time that it takes to type the commands into the buffer is usually more than made up for by reductions in subsequent errors and typing.

Counting the \C's:

A stream directive like "\B" or "\N" is a *single character* as far as FRED is concerned. Certainly, you type a stream directive as a backslash followed by a letter; however, as soon as FRED sees that kind of combination, he converts it into a special internal representation that is a single character.

This is important to remember when you are writing buffers, because sooner or later you will want to place a stream directive construction into your buffer. Suppose, for example, that you want to put a blank line in front of every line that begins with a "%" character, and then you want to delete the "%". This will take an I command to insert the blank line, and then an "s/%//" to delete the "%". You begin to type in your buffer with

```
a
/^%/i

\F
```

The A command begins to append text to the buffer you are creating. You type in the I command, the blank line, and the "\F" which ends the I command's input. However, FRED thinks that you are typing in the "\F" to

end the A command's input! How do you get FRED to realize that the "\F" is part of your text, not an indication that you are finished with text input?

You will recall that the "\C" character was used to turn off the special meanings for characters in FRED. In a pattern for example, "\C$" stands for the "$" character, instead of the end of the line. In the same way, putting a "\C" in front of "\F" tells FRED that you want to treat "\F" as an ordinary character, not as the special character that indicates the end of text input. Thus you would type

```
a
/^%/i

\C\F
+1s/%//
\F
```

If you now list the buffer, it will contain

```
/^%/i

\F
+1s/%//
```

which is exactly what you want. (Note that you would NOT get the same results by typing "\\F". "\\F" gives you a backslash character followed by an "F", not the single "\F" character.)

The same principle applies when you wish to enter other stream directive constructions into a buffer. For example, suppose you want to calculate the number of completely blank lines in buffer "0". You could do this with the program

```
b0 g/^ *$/
n(xx):#
jm;There are \N(xx) blank lines in the buffer.;
```

(Remember that the G command sets the count register to the number of line matched by the pattern.)

If you typed the above JM command as

```
jm;There are \n(xx) blank lines in the buffer.;
```

FRED would replace the "\N(xx)" as soon as he saw it. If "xx" contained the number 5 say, your buffer would contain

```
jm/There are 5 blank lines in the buffer./
```

and you would always get that message out when the program was executed. What you want in the buffer is a "\N" character. Thus when you type in your commands you must type

```
jm/There are \C\N(xx) blank lines in the buffer./
```

The "\C\N" turns into a single "\N" character, which is what you want.

This is all relatively straightforward, though it is easy to forget when you aren't careful. Things get a little trickier when you use Global and Until commands. Remember that these commands copy their command lists into a private buffer and then execute the commands in that buffer. For example, suppose you wanted to make a Global statement that would not only find blank lines in a buffer, but would tell you the line numbers of the blank lines. At first, you might think you could write this as

```
g/^ *$/n(bl)a jm:Line \N(bl) is blank.:
```

This G command will only execute its command list on blank lines. The N command assigns the line number of each blank line to the number register "bl", and the JM command prints a message. However, if you try to execute the above command, you will find that you get a lot of messages of the form

```
Line 0 is blank.
```

What has happened? When the list of commands was copied into the hidden buffer, the "\N(bl)" was *immediately* replaced with the value in the number register "bl" (probably zero). Thus the JM command that was executed repeatedly was

```
jm:Line 0 is blank.:
```

not

```
jm:Line \N(bl) is blank.:
```

To get the proper JM message in the Global command's buffer, the command must have the form

```
g/^ *$/n(bl)a jm:Line \C\N(bl) is blank.:
```

This will give a number of different messages like

```
Line 34 is blank.
```

Next, how would you type this command into a buffer? We have pointed out above that typing "\C\N" while appending text just gives you a "\N" in the buffer. To enter a "\C" and a "\N", you need a "\C" for the "\C" and another "\C" for the "\N". Thus if you wished to enter the proper command into a buffer, you would type

```
g/^ *$/n(bl)a jm:Line \c\c\c\n(bl) is blank.:
```

If you now print the line from the buffer, FRED will show

```
g/^ *$/n(bl)a jm:Line \C\N(bl) is blank.:
```

This is a good time to make a few points about the cases of letters in escape sequences. You can type the "\C" character with either an upper or lower case "C". However, as soon as FRED sees either "\c" or "\C", he will convert it to his special internal representation; from this point onwards, the character will always print out as "\C" since FRED always displays these special characters with upper case letters. The same principle holds for all other stream directive characters.

Getting the right number of "\C" characters in your commands does not have to be difficult; if you think things through carefully, you should have no trouble. However, we should warn you that you can sometimes need a lot of "\C" characters to make things right. To finish off this section, we illustrate one such problem.

Suppose you are getting an essay ready for TFing. Each paragraph in your essay is currently indented by five blanks. You want to remove those five blanks and instead put a temporary indent command ".ti +5" at the start of every paragraph. To do this for one paragraph, you could use the command

```
s/^     /.ti +5\C
/
```

This replaces the five blanks at the start of the line with ".ti +5" followed by a carriage return. The "\C" is necessary in front of the carriage return so that FRED knows there is more to come in the command. If you were going to put this in a G command, you would have to type

```
g/^      / s//.ti +5\C\C\C
/
```

The first "\C\C" gives you the "\C" character you need in the S command. Next you want a carriage return, but you need a "\C" in front of that too, again to tell FRED that there is more to come. Finally, if you were typing this into a buffer you would have to type

```
g/^     / s//.ti +5\c\c\c\c\c\c
/
```

For every "\C" you want in the command, you have to type two when appending the command to a buffer.

If you find you need too many "\C" characters, it is quite possible that you are making things hard for yourself. For example, we could have avoided some of the above problems by using the command

```
*s/^     /.ti +5\C
/
```

instead of a Global command. With a little thought, you can often eliminate a lot of "\C" problems by going about things a different way.

Other Stream Directives:

There are several other stream directives that you may find useful when writing buffer programs.

"\L" is similar to "\B" in that it places the contents of a buffer into the input stream. The difference is that every character in the buffer is treated as if it is preceded with a "\C". In one of our examples, we will show how this can be used.

"\S" is similar to "\L". The difference is that every new-line character in the buffer is removed before the buffer is placed in the input stream.

The "\R" construction is handy when a buffer program needs input from the user. When FRED sees a "\R" character, he diverts the input stream to the terminal to obtain one line of input. No stream directives typed in on this line are effective. The new-line character on the end of the input line is discarded. A very simple way to use this feature is shown in the following buffer.

```
jp/What file do you want me to read?/
br *d a \R\F
b(file) r \S(r)
```

This uses a JP command to ask the user for a file name. Buffer "r" is cleared with a "*d" and then the user's answer is appended to this buffer. The final statement switches to buffer "file" and reads in the desired file by using the "\S(r)" construction to obtain the file name from buffer "r". We could also have used "\B(r)" or "\L(r)" in this case, since all three stream directives would have the same effect.

The final stream directive we will discuss is "\E". The E command is used to define and name a pattern in FRED; for example,

```
e(d)/^[0123456789]/
```

defines a pattern named "d". (Pattern names follow the same rules as buffer names and label names.) This pattern will match any line that begins with a digit (recall that the pattern consisting of a number of characters enclosed in square brackets matches any of the characters in the brackets). Once the pattern has been defined in an E command, you can use it inside patterns with the stream directive "\E(<name>)". For example

```
/\E(d)/d
```

will delete the first line it finds that begins with a digit. Note that the "\E" stream directive is only applicable inside FRED patterns.

There are three or four other stream directives that are recognized by FRED, but we won't discuss them here. For complete details, see the FRED Reference Manual [2].

Comments

It is always a good idea to put comments in your programs, whether the programs are written in FRED or in some other language. Comments in FRED are written using the double quote character. When FRED sees a double quote in any command other than a JM, JP, or JE message, he ignores everything from the double quote to the end of the line. For example,

```
"this is a comment
*d "this clears the buffer
"this *d doesn't do anything
```

In a buffer program, you quite often want to go to a line of text without printing that line out. This can be done using the double quote with a line number. For example,

```
1"
```

goes to the first line of text but does not print it out. If you just typed the 1 without the double quote, the first line would be printed. Another way of doing this is to use the Z (Zap address) command. For example,

```
3z
```

sets the current line pointer to line 3 without printing anything.

Read Lists

Sometimes a FRED buffer wants to read a file but isn't sure of the file's name or location. To handle situations, FRED allows you to specify a list of possible file names on the R command, as in

```
r file1,file2,file3
```

When FRED finds an R command like this, he begin by trying to read the first file in the list. If this read is successful, FRED goes on to the next command after the R. However, if the read fails because the file can't be found (or because there is a syntax error in the file name), FRED tries to read the second file in the list. FRED keeps on trying file after file in the list until he finds a file or he gets to the end of the list.

If any of the read operations work, the condition register will be set to TRUE and the count register to the number of blocks read. If none of the read operations work, the condition register is set to FALSE.

Note that FRED only tries a new file on the list if the previous file could not be found or its name had a syntax error. If there is any other error in the read operation (e.g. permissions denied), FRED does not try any more files. FRED simply sets the condition register to FALSE and goes on to the command after the R.

As a special case, you can put a trailing comma on the end of the file list, as in

```
r file1,file2,file3,
```

This tells FRED to set the condition register to TRUE even if he can't find any of the files on the list. If FRED goes right through the list and doesn't find any of the files, the condition register is set TRUE and the count register is set to -1. Remember that if a read operation fails for some reason other than "file not found" or "file name syntax error", FRED never gets to the end of the list and the condition register will be set to FALSE.

As an example of how this could be used, let's go back to a buffer that we wrote early in this guide. We originally wrote it as

```
r /file1 j(go)t
r /file2 j(go)t
jm:Can't read either /file1 or /file2.:
q
@(go)
"More commands
```

We can now shorten this to

```
r /file1,/file2
jt je:Can't read either /file1 or /file2.:
```

As another example, suppose a particular buffer allows users to have a file named "options" containing FRED commands that set options for the buffer program. This file could be stored in a quick access file named "/options", a temporary file named "options", or a file in the "/fred" catalog (/fred/options). To execute the such commands, you might write

```
b(options) *d r /options,options,/fred/options,
jt je:Error reading options file:
\B(options)
"More instructions
```

Not only does this give you the chance to search for the file in a number of places, but it also gives you the opportunity to indicate the order in which FRED should search for the files.

While we're looking at these two buffers, we might say a little about how to produce informative error messages. If we took the line

```
jt je:Error reading options file:
```

and changed it to

```
jt jp'Error reading options file: ' y je
```

we can use the Y command to give us a more detailed report of what the actual error was. Looking at the previous program, we might have written

```
r /file1,/file2
jt jp'\W:' y je
```

This prints the name of the last file FRED tried to read, followed by whatever Y provides as the reason for the error.

Since this technique always prints the name of the *last* file that FRED tried to read in the read list, you may want to adjust the read list slightly to provide a more informative error. For example, suppose that the person who is using the program is supposed to enter the name of a file and the file may be in a quick access file or under the catalog "/fred". You might write

```
jp?Please enter file name:?
b(file)a
\R
\F
b(in) r \S(file),/fred/\S(file),\S(file),
jt jp?Error on \W:? y je
```

This first looks for the file under the name that the user gave, then looks under "/fred". If the file isn't under the catalog, it tries again under the original name. Of course, the file still won't be there, but when the error message is printed out, it will give the name that the user typed in (which will be more easily understood).

Executing FRED Buffers

There are two simple ways to execute a FRED buffer. If you are already in FRED and you have the buffer available, you can execute the commands in it just by typing

```
\B(buffer name)
```

at command level. FRED will fetch commands from the specified buffer and will execute them one by one. When it runs out of commands, it will revert to taking commands directly from the terminal (provided of course that the buffer didn't do anything drastic like Quitting FRED while it was executing).

If you're at system level and you have a buffer program stored in a file, you can execute the program with a command of the form

```
fred filename arg1 arg2 arg3 ...
```

When FRED sees this kind of command line, he reads the contents of the specified file into a buffer named "b(.)". The arguments that are given on the command line are placed one per line in buffer "b(0)". Your buffer program can obtain these arguments from "b(0)" as it executes. As a service to the user, FRED also places your userid in a buffer named "b(u)", the date in a buffer named "b(d)", and the current time in a buffer named "b(t)".

You don't have to specify the full filename of the file that contains the buffer. FRED will try to read the given filename first; if this doesn't work, he will look under your userid/fred/filename; and it that fails too, he will look under

```
library/fred/filename
      and
fred/filename
```

In essence, he places the filename into a buffer we will call "f" and then performs the sequence of commands

```
b(.)
r \S(f),\S(u)/fred/\S(f),library/fred/\S(f)/,fred/\S(f)
jt je/? could not access \B(f)/
@(a) "and so on
```

Note how we have used the userid that is stored in buffer "b(u)" in the construction "\S(u)".

We should point out here that FRED will execute your FRED init file before it begins execution of a buffer program. We will say more about init files when we get to our buffer examples.

Once FRED has executed a program called in this way, it will quit with QQ.

Options

Most buffer programs require that some FRED options be turned on and some be turned off. Because buffers tend to be used for complicated processes, they often use options that the normal user does not have turned on by default. For this reason, your buffers will often have to set up their own option environments before beginning execution.

In the examples which follow this section, we will always assume that FRED begins with the standard default options set. When we need to change these options, we will do so explicitly.

If you are writing a buffer that may be used by other users besides yourself, you should bear in mind that they may have init files which set different default options than your own. Because of this, you will usually have to set all the options you need explicitly at the beginning of your buffer.

The stream directives "\C" and "\O" may be used in certain cases to force special options to be disabled or enabled. A "\C" in front of any special pattern character means that the character is to be taken literally, regardless of any special meaning it might have because of the options. For example, suppose you wish to test for a "{" in a line. If the user has specified the "O+S{" option, the command

```
t/{/
```

will receive a syntax error because the special character "{" is used incorrectly in the pattern. To circumvent this, you could say

```
t/\c{/
```

This will test for a "{" character regardless of whether the "O+S{" or "O-S{" option is in effect. The "\C" tells FRED to take the "{" literally.

In the same way, "\O" in front of a character tells FRED to interpret that character with any special meaning it might have in this context. For example, the command

```
s/abc/&&/
```

will turn "abc" into "abcabc" if the "O+S&" option is in effect; however, it will turn "abc" into two ampersands if the option is not in effect. A "\O" in front of a character turns on its special meaning, regardless of the way options are currently set. Thus

```
s/abc/\O&\O&/
```

will turn "abc" into "abcabc" whether "O+S&" or "O-S&" is in effect. Thus, "\C" and "\O" provide another means of circumventing differences in option settings.

There is one important option that we should mention before we begin our examples. The Monitor option is turned on with the command

```
O+M
```

This option instructs FRED to monitor all work done during the execution of buffers. This includes work done in the hidden buffers used by the G and U commands. In this way, FRED provides a trace of what commands were executed from the buffer.

The Monitor output is printed to the terminal and is a character by character reproduction of the commands and text used by FRED during buffer execution. Comments and commands skipped by jumps are NOT printed. Generally speaking, Monitor output is printed with many commands on the same line. This can be a little messy, but it's also quite useful when you are trying to debug your buffer programs.

The O-M command turns off Monitor output.

Buffer Program Examples

We will now give a number of examples of FRED buffer programs. The programs deal with a wide variety of applications, some more specialized than others. Whether or not you will be able to use any of these buffers in your own work, they will at least demonstrate how FRED commands can be combined to perform specific tasks.

Note that when we present a buffer program, we will show how it should look once it is inside the buffer. When you are appending these commands to a buffer, don't forget that you will have to put "\C" characters in front of every stream directive we show here so that all the special characters are entered correctly.

Example 1: Shrinking a Card File

The first example deals with a file that has been created by reading in a deck of cards [4] containing a Fortran program which is to be prepared for use on the Time Sharing System. The problem is that there are many blanks in the file. In fact, each line has 80 characters, regardless of the length of the Fortran statement. We want to remove the trailing blanks and check that any continuation card has an ampersand (&) in column 6.

This buffer is one which can be invoked at command level (that is in response to the prompt "*") by means of a command like

```
fred shrink infile outfile
```

where "shrink" is the name of a file containing the following buffer. "infile" is the name of the file containing the Fortran program and "outfile" is the name of the file where the shrunken source code should be stored. If "outfile" is not specified, the buffer will use the workfile "*src" to hold the shrunken code; if "infile" is not specified either, the buffer assumes that the un-shrunken code is stored in "*src". Remember that FRED's bootstrap loader places the arguments on the command line on separate lines in buffer "0". For this reason,

one of the first things the following buffer does is check how many lines there are in buffer "0" to see how many files were specified on the command line.

```
o+s{              " set options for pattern tagging
o+q               " set Quit option
b0                " move to buffer 0
n(xx):$           " assign the number of lines in
                  "   buffer "0" to register "xx"
n(xx)>1           " set condition code to "true" if
                  "   the number is greater than 1
j(a)t             " jump to label a if "true"
$a *src
*src
\F                " append two lines with "*src" after
                  "   the last line of the buffer
@(a)              " a label
1m(f.in)          " move the name of the input file
                  "   to buffer "f.in"
1m(f.out)         " move next line to buffer "f.out"
*d                " delete all remaining lines
                  "   in buffer "0"
r \S(f.in)        " read input file
jt y je//
g~/^c/ s/^{.....}t[^ ]/t\C\C&/
                  " if not comment, change continue
                  "    character to &
*s/  *$//         " remove blanks at the end
                  "   of each line
w \S(f.out)       " write out shrunken file
jt y je//
q                 " all done
```

The read statement

```
r \S(f.in)
```

assumes that buffer "f.in" contains the name of a file that is to be read. If the read is successful, the condition register is set to TRUE. The JT on the next line will skip over the Y and the JE and execution will continue to the G command. On the other hand, if there were any errors detected while attempting to read (e.g. trying to read a nonexistent file) the condition register is set to FALSE. The Y command will then be executed to tell the user what went wrong, and the JE will stop all execution.

The statement

```
g~/^c/ s/^{.....}t[^ ]/t\C\C&/
```

first locates all lines that do not begin with "c" (comment statements). The S command then causes any five characters at the beginning of the line (the tagged expression {.....}t) followed by a non-blank character to be replaced by exactly those same five characters followed by an ampersand. The "\C" must appear because of the special role of the ampersand in the substitution. Since you want an ampersand character and not the string that matched the pattern of the S command, you must have a "\C&" when the substitution is made. Because the list of commands for the G command are *appended* to a hidden buffer, you must write "\C\C" to put a single "\C" into the hidden buffer. When typing this command into the buffer, remember that you must type *four* "\C" characters to get two out.

The "*s" in the statement

```
*s/  *$//
```

means that the substitution is to be attempted in every line. The pattern " *" will match any string of zero or more blanks and the "$" matches the null (imaginary) character before the carriage return at the end of the line. Hence, the pattern " *$" matches any string of one or more blanks at the end of a line. The above command thus removes all the blanks at the end of a line by replacing them with nothing. An alternate form that will also work is "1,$s/ *$//".

You can verify that the unwanted blanks have been removed from the file by making the substitution "*s/ /%/", displaying the buffer by means of the command "*p", and then changing the "%" back to blanks by means of the command "*s/%/ /". Here, we have used the "%" because it is a character that is seldom used in Fortran source code. You can use any other character that does not already appear in your buffer for this test. When changing back to blanks, any occurrences of the "%" which might already have been present will be replaced by blanks. If you want to remove all the blanks (good grief!) from the buffer, try this: "*s/ //".

You are probably wondering how you can tell if a particular character or string is present in the buffer. The statement

```
g/%/p
```

will print all lines containing a "%". Similarly, the statement

```
g/CALL/p
```

will print every line that contains the string "CALL". To find out how many instances of the string "CALL" there are, you could use

```
*s/CALL/&/  #
```

As we have mentioned previously, the S command sets the count register to the number of successful substitutions made.

The write statement "w \S(f.out)" writes the contents of buffer "0" to the file whose name is in buffer "f.out". If an error is detected in writing, or if a file has to be created, the Y command will be executed to tell what the problem was and execution will stop.

There is a variation on the above buffer which reduces the number of FRED statements for this example at the expense of having a slightly longer algorithm. This variation simply omits the two N commands and the "j(a)t" at the beginning of the program, and automatically appends two lines containing "*src" to "b(0)". The end result is the same.

There is a second variation on this example for users who would prefer to use the input file as the default output file if only one file name was specified. This is done by replacing the three lines immediately following the "j(a)t" with the commands

```
$a *src
\F
1k(.w) za(.w)
```

This appends a copy of the first line of "b(0)" to the end of "b(0)". Thus if only one file name was specified on the command line, that file will be used for both input and output.

You may find that one of the above conventions of having three different forms of the same command is useful in other contexts. Of course, the G and S commands may be replaced by whatever statements you would like to execute.

Example 2: A Memo File

In this example, we will show how to create a memo file. Each message is entered at the beginning of the file so that the most recent memo comes first (last in, first out). Each memo carries the date and time that it was entered. This set of statements can be typed directly from your terminal.

```
b(.w)  *d  r /memos
0a ***** \S(t) \S(d)
  .
  . your message
  .
\F
w /memos
```

The second line begins to insert text at the beginning of buffer "b(.w)" (i.e. after line zero, before line one). The opening line of the new memo contains a string of asterisks as a decoration and the date and time. Note how we have made use of the buffers "b(t)" and "b(d)" which are provided by the FRED bootstrap; "b(t)" contains the time that FRED was entered and "b(d)" contains the date. After this opening line, you may enter any number of lines of text as a memo message. Input stops when you type a "\F".

It would be convenient if we could cause these steps to take place without having to type each of them every time. What we will do is prepare a buffer named "m" which will contain the instructions needed to do the job. To add a memo to a file of memos or to create a file with a memo, all we will have to do is type

```
\B(m)
```

This will cause all of the FRED commands in buffer "m" to be executed. Our buffer "m" will contain the following commands.

```
b(.w)  *d  r /memos
0a ***** \S(t) \S(d)
\F
uf  .a \C\R\C
\F .t/./
w /memos
```

This buffer starts by reading file "/memos" (which need not exist) into the empty buffer ".w". An opening line is placed at the start of this buffer to mark the beginning of the new memo.

The UF command which begins in line four carries over to line five because of the "\C" in front of the carriage return; the "\C" indicates that the carriage return isn't the end of the until loop, it is just a character that is used in the loop. The UF command begins by creating a hidden buffer and *appending* all of the statements following it up to the next carriage return that is not preceded by a "\C". Thus the last statement in the until buffer is ".t/./". The "\C" is needed before the "\R" for the usual reasons when using stream directives in G and U commands. The hidden buffer for the UF command will therefore contain

```
a \R
\F .t/./
```

The T command sets the condition register to TRUE if any characters were entered as input for the "\R". If a carriage return was the only thing entered, the condition register is set to FALSE and the UF loop terminates. Thus to indicate the end of your memo, all you have to do is type in a null line (i.e. just a carriage return).

Example 3: A Write Buffer

One of the most unhappy experiences of computer users is accidentally writing over an important file with garbage that you didn't intend to put there. One way to avoid this is to adopt a convention by which every file contains the pathname under which it should be stored. One convention which has been found convenient is to have the first or second line of every file take the form

```
comment === pathname ===
```

The equal signs serve to set off the pathname which identifies the file to which the buffer is to be written. The "comment" can be any string which identifies a comment in the programming language being used (e.g. "C" in Fortran, "/*" in B, or ".cc ." in TF). For example,

```
C root finder === /FORTRAN/NEWTON ===
" a writing buffer === /FRED/WRITE ===
```

Suppose that you have been editing a buffer and wish to write it into a file. We will create a buffer "w" that will be executed by entering "\B(w)". If there is no line of the above standard form, the message "unable to write" is printed at the terminal. If there is a line of this form, the date and time are substituted after the second triple of equal signs and the buffer is written into the named file. If there is any error in writing or if a

file has to be created, the message "write error on <filename>" is given. Otherwise, the message "successful write on <filename>" is printed. Here is buffer "w".

```
o+s{
e(.w)/{^.*===}1 *{[^ ][^ ]*}2 *===.*$/
n(here)a                " save current line number
*t/\E(.w)/              " test for the presence of line
jt je/unable to write/
s/\E(.w)/1 2 === \S(t) \S(d)/
                        " insert the time at the
                        "   end of the line
k(fname)                " copy the line to b(fname)
u1 b(fname) s/\E(.w)/2/
                        " get the file name alone
\N(here)"               " restore current line number
wx \B(fname)
jt je/write error on \S(fname)/
jm/successful write on \S(fname)/
```

The first use of the expression ".w" is in line four where we look for the first line that contains the expression. If the condition code is not TRUE, the next line prints the message "unable to write". The expression is used again in line six, where anything following the second triple of equal signs is replaced by the contents of buffers "t" and "d". The line containing the file name is copied into "fname" with the K command.

The next command temporarily switches to buffer "fname" and does a substitution which reduces the full comment line into a line which only contains the required file name. This is put inside a U1 command to make use of one of the handiest features of the U command. At the end of a U-loop, the current buffer is always set to the buffer that was current before the loop began, no matter what buffer was current at the end of the loop. By putting the change to buffer "fname" inside a U-loop, we can manipulate the contents of "fname" and still return to the proper current buffer when our manipulations are finished. In this way, we can leave the current buffer and return without ever needing to know the current buffer's name.

Once we have the name of the file in "fname", we can write to that file using the "wx \B(fname)" command. WX is a simple variation of the usual W command; the W command does NOT print out the usual write statistics (number of blocks, lines, and characters written) when it is executed in a buffer, whereas the WX command DOES print out these statistics.

The only other thing we need to note about the above buffer is that it saves the current line number in the number register "here" and restores that line number as soon as the buffer manipulations are finished.

Example 4: Init Buffers

As you become more comfortable with FRED, you may find that there are some things you always do whenever you enter the subsystem. For example, you might set certain options that are not standard defaults or you may read in handy utility buffers like the write buffer we discussed in the last example.

You can have all these things done for you automatically by creating what is called an *init file*. Whenever FRED is called, either with the simple command "FRED" or a more complex command line of the form

```
fred filename arg1 arg2 ...
```

the editor looks for a file under your userid called "/fred/.init". The contents of this file will be read into buffer "b(f)" and will be executed before anything else happens. Once this has been done, the editor deletes buffer "b(f)". Then it either loads the required buffer program or begins to take commands directly from the terminal.

An init file can be as simple or ambitious as you want. For example, the following is a perfectly good init buffer.

```
o+s{([|
o+p
o+t5,9
o-i(
```

This just turns on a few useful pattern options, turns on paging, sets a few tab stops, and turns off some of the standard input options. For many people, this simple init file is all that's needed.

Another standard set of commands seen in many init buffers is

```
zd(t)zd(d)zd(u)
```

These are used by people who don't want the standard buffers provided by FRED containing the time, the date, and the userid of the current user. The ZD (Zap Delete) simply erases whatever is in the specified buffer, so that "zd(u)" erases buffer "u".

Of course, it is possible to create an init buffer that does a lot more than the simple things described above. Many people create init buffers that actually simulate the actions of the standard FRED bootstrap loader. Below we will examine one such buffer program in detail. This program can be found in "fred/sample/.init2". Note that we have expanded it, included comments, and made one or two minor changes to make the buffer more readable. There are also several small differences between the behavior of this buffer and the behavior of the actual bootstrap loader.

```
o+p o+s{|(^.$*[     " useful options
b(0)a "\R
\F                  " append command line to b(0)
k(.)                " copy command line to b(.)
s/  */\C
/                   " break up command line
g/^$/d              " get rid of null lines
1d                  " get rid of "fred"
*t/./               " is anything left?
j(b)t               " if there are args, go to @(b)
                    " otherwise delete junk and
                    " get ready for normal editing
o-q u1 zd(u)zd(d)zd(t)zd(.)zd(f)zd(0)je/Fred/
@(b)                " you get here if there were args
t/^r$/              " is this a read command?
j(r)t               " if so, go to @(r)
m(g)                " otherwise, move file name to b(g)
                    " attempt to read file into b(.)
b(.)$r \B(g),\B(u)/fred/\B(g),fred/\B(g)
                    " error if you haven't found
                    "     anything by now
jt je;? could not access \S(g);
                    " you get here if read worked
zd(g)               " get rid of file name
b(0) o-q
u1 zd(f)\C\B(.)je//
                    " delete init buffer, execute
                    " stuff in b(.), and exit
@(r)                " you get here if command was "r"
1d                  " delete "r" in b(0)
t/./                " is there anything to read?
                    " if not, error
jt je/? read what?/
1m(g)               " otherwise, move filename to b(g)
b(f) $-2za(0)       " append args to b(f)
b(0) *d             " clear buffer zero
rx \B(g)            " read in file
                    " if error, say why and exit
jt y je//
$-&&t/^\C$ |[^    ]*  \C (ld[aqx]|end|octal)  /
                      " is this a GMAP program?
                      " if so, set GMAP tabs
                      " and say you've done it
jf o+t8,16,32,72 jp/o+t8,16,32,72/
```

```
                    " say you're printing line 1
jm/1p/
1p                  " and print it
o-q                 " delete junk and exit
b(0) u1 zd(u)zd(d)zd(t)zd(g)zd(.)zd(f) je//
```

This buffer does a lot of things and covers a number of situations. We will describe the actions of the buffer in very general terms, and leave it to the reader to figure out the details from the comments given above.

First the buffer sets some options. Here is where you can specify other options if you wish. It then appends the entire command line to buffer zero and copies the command line to "b(.)" with a " in front of it so that it will be treated as a comment if "b(.)" is executed. The command line in "b(0)" is broken up by changing every string of blanks into a carriage return. Line 1 contains the word "fred" from the command line and this is deleted along with any null lines. If buffer zero is now empty, the command line only consisted of the word "fred"; therefore this is just a simple editing session and unneeded buffers can all be eliminated. The "je/Fred/" commands prints the familiar "Fred" acknowledgment.

If there was more to the command line than just "fred", the buffer sees if the next string was a single "r". If not, the next string is taken to be a file name. The file name is copied to "b(g)" and the init buffer makes several attempts to read the file. If the attempts fail, it prints an error message; otherwise, it reads the file into "b(.)" and then executes the buffer with a "\B(.)" construct inside a U1 loop. The U1 loop is used for the same reason that we used the U1 in the previous example.

If the command line had the form

```
fred r filename
```

the buffer will find its way to the label "@(r)". It checks to see if a "filename" really was specified, and if so, attempts to read the file. If the input file looks like GMAP source code, standard GMAP tabs are set for the file. The buffer displays the first line of the file, cleans up unneeded junk, and prepares for normal editing.

There are several ways to modify this init buffer for your own purposes. One of the easiest ways is to change the options at the top of the file to the options you want to use. Another simple change is to have the buffer print the last few lines of your file rather than the first line.

We might point out that you can use the "fast read" command line

```
fred r filename
```

even if you don't have this feature built into your init buffer. The reason for this is that there is a buffer stored under "fred/r" that will do the fast read for you. This read buffer is very similar to the init buffer we have just discussed. However, you don't have to understand it to be able to use it. Of course, the same applies for the above buffer; provided that you set the right options in the first line, you can use the init buffer above with a high degree of safety, even if you don't understand every line of it.

Example 5: A Buffer to Generate Buffers

In this example, we give ways in which a buffer may be stored in a file and invoked to set up a number of other buffers for you to use during a work session.

A simple and direct way to do this is to keep your various buffers in distinct files, one buffer to a file. For example, your buffer to create buffers might contain statements like

```
b(name1)  r /path1
b(name2)  r /path2
```

If you put this buffer into a file called "/start" (or "/fred/start") and invoke FRED by the command

```
fred /start
```

file "/start" will be read into buffer "b(.)" and executed.

Alternatively, you could put your buffer to make buffers into your init file "/fred/.init" and it will be executed every time you enter FRED.

Both these approaches work well. However, you might wish to keep several of your buffers in a single file. If so, you could have a file something like this:

```
o+s{}
o-i(
b.   /^>b/"
uf .k~ b~ t/^>b./ jf s/ .*// s;^>b;.,/^>b/-1m; b. \C\B~
b0  zd~  u1  zd.  je/Ready:/
>b(name1)  comment 1
     .
     .
     .
>b(name2)  comment 2
     .
     .
     .
>b
" end of file
```

The third line tells FRED that you want to work on "b(.)". Remember though that the bootstrapper will be loading .bf this file into buffer ".", so it will be acting upon itself. This buffer cuts itself into several buffers. When doing this kind of auto-surgery, you must be careful to obey the following rules:

1. Do not alter the line being executed.
2. Do not append text after the last line.
3. If the buffer that is currently executing was called from another buffer (e.g. by a "\B" in the calling buffer), do not alter the current line in the calling buffer or the line after the current line, and do not append text to the end of the calling buffer. The same principle applies when a buffer calls a buffer which calls another buffer and so on.
4. You must not change the current line in any of the calling buffers or append text to the bottom of those buffers.

The third line also sets the current line to be the first line starting with ">b". The UF command creates a hidden buffer containing these statements:

```
.k~ b~ t/^>b./ jf s/ .*// s;^>b;.,/^>b/-1m; b. \B~
```

The current line is moved into buffer "~" and buffer "~" becomes the current buffer. If there is no character following the ">b", the T command sets the condition register to FALSE and a jump is made to the end of the until buffer; the condition register will still be FALSE, and therefore the UF loop will terminate. If there is something after the ">b" (representing a buffer name), the substitutions delete the comments and replace the string ">b" by the string ".,/^>b/-1m". Buffer "~" will then contain the line

```
 .,/^>b/-1m(buffname)
```

This is executed on buffer "." causing some of the lines of buffer "." to be moved to buffer "buffname". This process continues until the line with exactly ">b" is found by the T command. Since this line does not match the expression "^>b." (there is no character after the "b"), the condition register is set to FALSE and the UF loop stops executing. As you can see, many buffers can be created this way.

The next line switches to buffer zero and deletes buffer "~". It then deletes buffer "." while executing from the hidden buffer created by the U1 statement. This is somewhat like being out on a limb and cutting down the tree, which may seem like a dangerous thing to do. There is no disaster because the JE statement halts the execution of all buffers and prints the message "Ready".

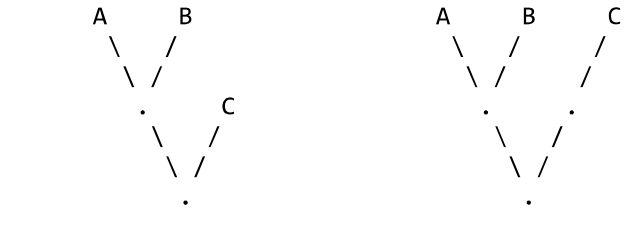
This buffer was designed to operate out of buffer "."; you could put it into a file "start" and use it in the way described in the previous example.

If you would like to have the same effect but prefer to work out of the file "/fred/.init" you could replace the first few lines by these:

```
o+s{ o-i(
bf  /^>b/"
uf .m~ b~ t/^>b./ jf s/ .*// s;^>b;.,/^>b/-1m; bf \C\B~
b0  zd~  jm/Ready:/ u1 jo
```

Now the auto-surgery is being done on buffer "f". Notice that in line three there was only one reference to buffer "." and this has been changed to "f". Note also that the "zdf" that might have been in line four is not needed because this is done by the bootstrap.

Here is a buffer that determines whether two rooted trees are isomorphic. We use bracket notation to represent the trees. For the definitions of "tree", "rooted tree", and "isomorphic" consult any good book on graph theory such as "Graph Theory with Applications" by J.A.Bondy and U.S.R.Murty (published in 1976 by Macmillan of London and Elsivier). Here are two non-isomorphic rooted trees and their bracket representations.



Here is buffer "trees".

```
" === /fred/trees === ljdickey 08/28/1980 17:20
" test contents of buffer "0"
"                  two isomorphic rooted trees
" the brackets may be of these types:
"                      ( ),  < >,  { },  or [ ].
o+s[ o-i(
b0  *k1  b1  *s/[({[]/</  *s/[)}\C]]/>/
*s/[^<>]//  *s/<>/<:>/  *s/./&\C
/  *s/\C
//
uf  $-/^<$/,/^>$/m2  b2  1d  $d  *zs\C
     b1  i <\C\L2>\C\F  1t/^<$/
$n(tt)a  n(tt)=2  jt  je/** Not 2 trees./
1m2  t/^\L2$/  jt  je/The trees are NOT isomorphic./
je/The two trees are isomorphic./
```

If you've read this far, it's clear that you're on your way to becoming a buffer duffer and will enjoy working out an explanation of each of the steps for yourself. You will see that a copy of the data stays in buffer "0", that all of the work is done in buffers "1" and "2", that all brackets are replaced by "<" and ">", and that every character is placed on a line by itself without any carriage return. You will have to pay close attention to the use of the "$" symbol that matches the null (imaginary) character at the end of each line, even when there isn't a carriage return at the end of the line. The heart of the method lies in using the sort "*zs" to put the representation into a canonical form.

Acknowledgments:

The authors wish to express appreciation to Michael A. Afheldt, William C. W. Ince, and Peter J. Fraser for their kind assistance in reading the preliminary versions of this booklet and suggesting changes which improved it.